UDLAP®

Universidad de las Américas Puebla

Práctica 3:
Contador ascendente–descendente de 9 bits con signo

Profesor:
Gerardo Ulices Díaz

Integrantes:
Diego Camargo – 180892
Pablo Robles – 166855
Bruno García – 177613

Materia:
Sistemas Embebidos

Entrega:
20 de octubre de 2026

Resumen

En este reporte se implementa un contador ascendente/descendente de 9 bits con signo desde −256 a 255 en la tarjeta Terasic DE10-Lite. El sistema separa una unidad de control, una FSM de tres estados (pausa, ascender y descender), de una ruta de datos formada por el registro del contador. Para las entradas mecánicas y evitar su margen de error se ajusta con un sincronizador de dos flip-flops, filtro antirrebote de 10 ms y un pulso de un ciclo. Para actualizar el contador se usó el *tick* como pulso de habilitación de 1 s o 100 ms generado a partir del reloj de 50 MHz. Los digitos del contador se muestran con signo en cuatro displays de 7 segmentos. Su funcionamiento se verificó en la tarjeta mostrando los tres estados, el cambio de velocidad y la representación de valores positivos y negativos en los displays.

Palabras clave: FSM, contador, 9 bits, display 7 segmentos, tick, clock, FPGA.

Objetivos

Diseñar e implementar un contador ascendente–descendente de 9 bits con signo, con limites en −256 y 255, con controles de reinicio, pausa, conteo ascendente y descendente y un ascelerador de velocidad de 1 s y 100 ms, mostrando el valor en cuatro displays de 7 segmentos de la DE10-Lite.

- Separar la unidad de control (FSM) de la ruta de datos (registro con signo).
- Acondicionar los botones con sincronización, antirrebote y pulso de un ciclo.
- Generar los intervalos de 1 s y 100 ms con un pulso de habilitación con un único reloj de 50 MHz.
- Verificar el comportamiento modular en los extremos del intervalo.

Marco Teórico

Representación con signo en 9 bits

En complemento a dos, los números negativos se obtienen invirtiendo los bits de la magnitud y sumando uno, y el bit más significativo actúa como signo [1]. Un número de 9 bits representa

$$-2^8 \leq x \leq 2^8 - 1 \;\Rightarrow\; -256 \leq x \leq 255, \qquad (1)$$

con un único cero (sin "cero negativo"). Los extremos son $255 =$ `0_1111_1111`, $-1 =$ `1_1111_1111` y $-256 =$ `1_0000_0000`. Al ser aritmética modular, $255 + 1 \to -256$ y $-256 - 1 \to 255$.

Máquinas de estados finitos

Una FSM es un circuito secuencial cuyo siguiente estado depende del estado actual y de las entradas, y cuyo estado se almacena en registros actualizados con el flanco de reloj. Aquí se emplean tres estados: PAUSA, ASCENDENTE (+1 por `tick`) y DESCENDENTE (−1 por `tick`). Esta unidad de control decide qué hacer, mientras que la ruta de datos (el registro de 9 bits) almacena y actualiza el valor [2].

Pulso de habilitación (clock enable)

Para obtener una actualización más lenta que el reloj, es preferible generar un pulso de un ciclo (*clock enable*) y usarlo como habilitación de registros del mismo reloj, en lugar de un reloj derivado. Así se mantiene un único dominio de reloj, se evitan los peligros de metastabilidad entre dominios y el análisis de temporización es más simple [6], [7]. El número de ciclos para un intervalo T con $f_{CLK} = 50$ MHz es

$$N = f_{CLK}\, T, \qquad (2)$$

es decir, $N_{1\,s} = 50{,}000{,}000$ y $N_{100\,ms} = 5{,}000{,}000$ ciclos.

Rebote y metastabilidad

Un pulsador mecánico produce varias transiciones rápidas al cerrarse (rebote), que un circuito síncrono leería como múltiples pulsaciones. Además, la señal del botón es asíncrona respecto al reloj; si cambia cerca del flanco puede violar los tiempos de *setup/hold* y llevar al flip-flop a un estado metaestable. Una cadena de dos flip-flops reduce la probabilidad de que la metastabilidad se propague, aunque no elimina el rebote, por lo que hay que dar un margen de error para evitar que se detecten estas pulsaciones. El filtro antirrebote exige que la señal permanezca estable un tiempo mínimo (en este caso≈10 ms) y se genera finalmente un pulso de un ciclo [2], [5].

Diseño del Experimento

Arquitectura

El sistema (Figura 1) se compone de tres módulos `button_onepulse` (pausa, incremento y decre-

mento), un `tick_generator`, la FSM con el contador (`signed_counter_fsm`) y un decodificador a 7 segmentos con signo (`signed_display_9bit`), integrados en `counter_9bit_top`.

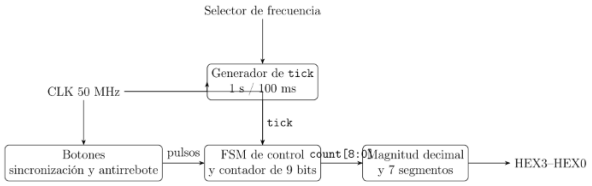


Figura 1: Diagrama de Bloques del Sistema

Diagrama de Estados

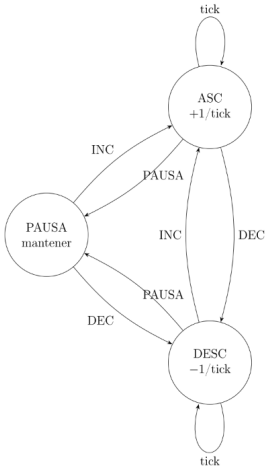


Figura 2: Diagrama de Estados. RESET lleva desde cualquier estado a PAUSA con cuenta 0.

Si dos controles coinciden en el mismo ciclo, la prioridad es PAUSA, luego INCREMENTO y finalmente DECREMENTO.

Entradas y salidas

Como se aprecia en el diagrama de estados, las entradas hacen las siguientes funciones:

Entrada	Función
`BTN_RESET_N`	Reinicio (cuenta 0, PAUSA)
`BTN_PAUSE_N`	Pasa a PAUSA
`BTN_INC_N`	Pasa a ASCENDENTE
`BTN_DEC_N`	Pasa a DESCENDENTE
`SW_FREQ`	0: 1 s 1: 100 ms

Cuadro 1: Entradas del sistema (activas en bajo).

Para ayudar a visualizar que entrada está siendo activada, se activaran LEDS que señalan el estado actual:

LED	Significado
LEDR0	Estado PAUSA
LEDR1	Estado ASCENDENTE
LEDR2	Estado DESCENDENTE
LEDR3	Velocidad rápida (100 ms)
LEDR9	Bit de signo

Cuadro 2: Asignación de los LEDs de estado.

Desarrollo del Experimento

Codificación de los módulos en Verilog

El código fuente completo se encuentra en el repositorio del proyecto (`https://github.com/CMRGO11/Contador-9-Bits`); se describe la lógica de cada bloque.

button_onepulse. Pasa la entrada por dos flip-flops y compara la muestra sincronizada con el último valor estable. Si difiere, cuenta 500,000 ciclos de 50 MHz (10 ms); si persiste, actualiza el valor estable y, solo en la transición 1→0 (pulsación), emite un pulso de un ciclo.

tick_generator. Contador de 26 bits que cuenta de 0 a $N-1$ (N = 50,000,000 o 5,000,000 según `speed_sel`) y emite un pulso de un ciclo. Se mantiene en cero cuando `enable` es 0, por lo que al reanudar transcurre un intervalo completo antes del siguiente cambio.

signed_counter_fsm. Registra el estado y el valor `count` de 9 bits con signo. Los pulsos cambian el estado con la prioridad indicada y, con cada `tick`, el contador suma o resta 1 según el estado. El reinicio lleva el sistema a PAUSA con 0.

signed_display_9bit. Obtiene signo y magnitud, separa unidades, decenas y centenas con división y módulo entre 10 y decodifica a 7 segmentos activos en bajo. HEX3 muestra un guion si el valor es negativo; HEX2–HEX0 muestran el valor.

counter_9bit_top. Integra los módulos, sincroniza `SW_FREQ` con dos flip-flops y asigna los LEDs según el *Cuadro 2*.

Implementación en Quartus

La implementación se realizó en Intel Quartus Prime [7] para el dispositivo MAX 10 de la DE10-Lite, con `counter_9bit_top` como entidad superior y reloj `CLOCK_50`. Como la tarjeta solo dispone de

dos pulsadores integrados y no se usaron pulsadores externos, únicamente el reinicio se asignó a un pulsador (KEY0) y los demás controles se asignaron a switches (Cuadro 3). Los displays HEX0–HEX3 y los LEDs integrados se asignaron a sus pines correspondientes.

Señal	Recurso	Función
`BTN_RESET_N`	KEY0	Reinicio
`BTN_PAUSE_N`	SW0	Pausa
`BTN_INC_N`	SW1	Ascendente
`BTN_DEC_N`	SW2	Descendente
`SW_FREQ`	SW3	Velocidad

Cuadro 3: Asignación física utilizada en la DE10-Lite.

Al ser interruptores y no pulsadores, el módulo `button_onepulse` sigue siendo útil: sincroniza y filtra la señal y emite un único pulso por cada transición de nivel activa (1→0), de modo que mantener el interruptor en una posición no repite el comando. `SW_FREQ` se lee por nivel (0: 1 s, 1: 100 ms).

Resultados Teórico-Práctico

Resultados de la simulación

La Figura 3 muestra la simulación del sistema con un `tick` abreviado para poder observar el recorrido completo del contador. En ella el sistema cuenta en estado ASCENDENTE (`state`=1), de modo que los bits de `count` conmutan en binario hasta que `count[7]` se activa a la mitad del recorrido (128 cuentas). Cerca del final de la ventana ocurren tres eventos consecutivos:

- 255 → −256**.** Al alcanzar 255 el siguiente `tick` activa `count[8]` y pone en cero los demás bits (`1_0000_0000` = −256): el contador se envuelve sin saturar.
- **Pausa y descenso.** Un `pause_pulse` lleva brevemente la FSM a PAUSA (`state`=0) y un `dec_pulse` posterior la pasa a DESCENDENTE (`state`=2).
- −256 → 255**.** En descenso, el siguiente `tick` desactiva `count[8]` y activa `count[7:0]`, es decir, `0_1111_1111` = 255.

La señal `reset` se activa solo al final de la ventana, por lo que la simulación muestra los tres estados de control y ambas transiciones de los límites, pero no la retención del valor durante varios `tick` en pausa ni el valor posterior al reinicio.

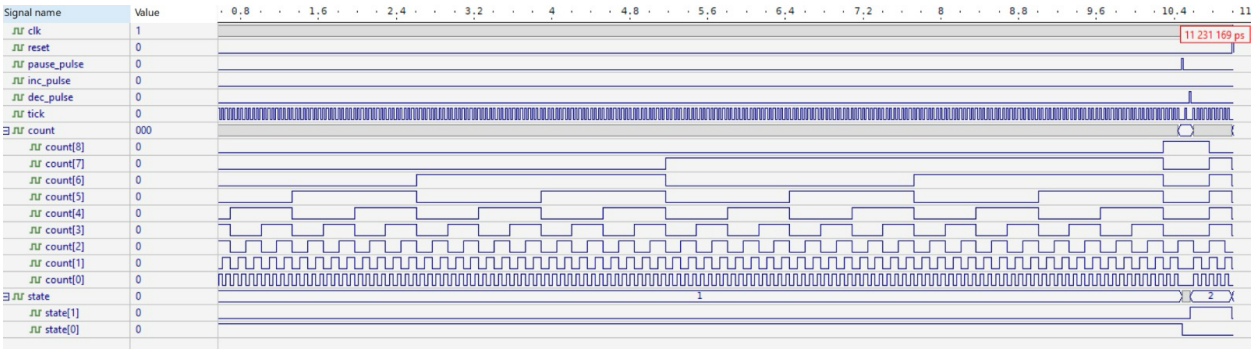


Figura 3: Simulación funcional: conteo ascendente, cruce 255 → −256, pausa, descenso y cruce −256 → 255.

Resultados de la implementación física

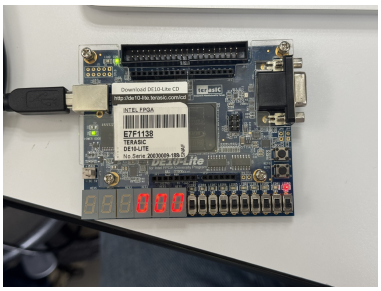
Se descargó el diseño en la DE10-Lite y se verificó cada función. Figuras 4 y 5 muestran las capturas.

Reinicio y pausa. Tras el reinicio el contador queda en 000 con LEDR0 encendido (Fig. 4a). En la Fig. 5c el contador está en pausa en −20: HEX3 muestra el guion, LEDR9 indica el signo negativo y LEDR0 la pausa.

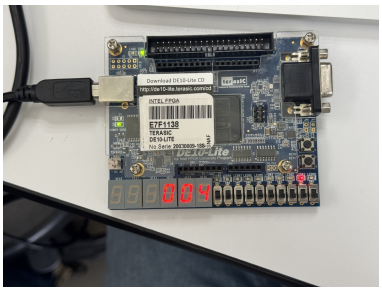
Ascenso. Con el incremento se enciende LEDR1; a velocidad lenta la captura muestra 004 (Fig. 4b) y a velocidad rápida (LEDR3 encendido) 022 (Fig. 4c).

Descenso. Con el decremento se enciende LEDR2; se muestran 040 a velocidad lenta (Fig. 5a) y 023 a velocidad rápida con LEDR3 (Fig. 5b). La cuenta continuó hacia negativos hasta −20 (Fig. 5c).

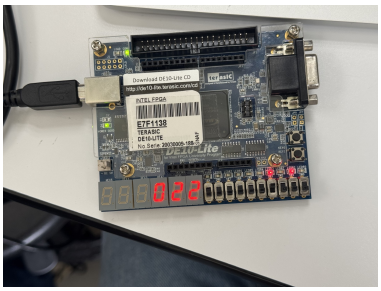
Comparación 1 s contra 100 ms. Con `SW_FREQ`=0 el contador cambia una vez por segundo (N = 50,000,000 ciclos) y con `SW_FREQ`=1 diez veces por segundo (N = 5,000,000), lo que la tarjeta indica con LEDR3.



(a) Reset: 000, LEDR0.

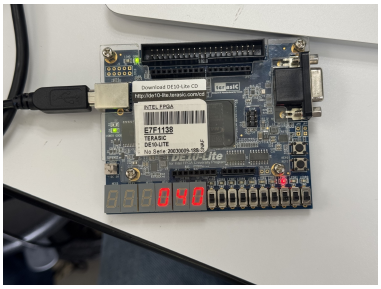


(b) Ascenso lento: 004, LEDR1.

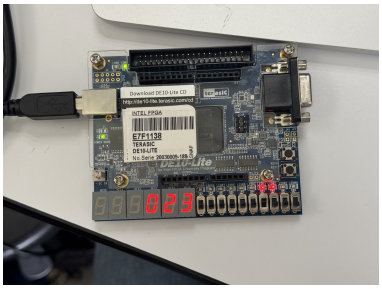


(c) Ascenso rápido: 022, LEDR1-R3.

Figura 4: Reinicio, ascenso y cambio de velocidad.



(a) Descenso lento: 040, LEDR2.



(b) Descenso rápido: 023, LEDR2-R3.



(c) Pausa en −20: LEDR9 y LEDR0.

Figura 5: Descenso, cambio de velocidad y pausa con valor negativo.

Dado las circunstancias del experimento, las imagenes no muestran perfectamente el funcionamiento del contador cuando reinicia, pausa, cambia de velocidad o las transiciones de 255→-256 y -256→255; por eso en el siguiente link se muestra un video mostrando su funcionamiento: `https://youtu.be/x82fZaSHalQ`

Conclusión

En esta práctica se diseñó, simuló e implementó un contador ascendente–descendente de 9 bits con signo en complemento a dos, cumpliendo la metodología de diseño digital (especificación → diagrama de estados → HDL Verilog → simulación e implementación).

En primer lugar, el uso de la representación en complemento a dos permitió codificar el intervalo de −256 ($\texttt{1_0000_0000}_2$) a 255 ($\texttt{0_1111_1111}_2$), eliminando la ambigüedad de un doble cero y unificando las operaciones de suma y resta en hardware. Los desbordamientos en los extremos ocurrieron perfectamente sin requerir comparadores adicionales: al sumar 1 a 255 se produjo la transición directa a −256, y al restar 1 a −256 el registro retornó a 255.

Asimismo, la separación arquitectónica entre la unidad de control (una máquina de estados finitos tipo Moore con estados `PAUSA`, `ASCENDENTE` y `DESCENDENTE`) y la ruta de datos (el registro acumulador con signo

de 9 bits y el decodificador de magnitud y signo a 7 segmentos) facilitó la modularidad, depuración y escalabilidad del sistema. Mientras la FSM gestionó la prioridad de las entradas ya acondicionadas (sincronizadas con doble flip-flop y filtradas contra rebotes mecánicos durante 10 ms), la ruta de datos se encargó exclusivamente de actualizar la cuenta y transformar su magnitud a BCD mediante división y módulo para los displays HEX3–HEX0.

Finalmente, se comprobó la importancia crítica de emplear un pulso de habilitación (*clock enable* o `tick`) de un solo ciclo en lugar de dividir el reloj mediante lógica combinacional o registros para crear nuevos dominios de reloj. Mantener todo el sistema sincronizado al único dominio de reloj global de 50 MHz (`CLOCK_50`) evita el sesgo de reloj (*clock skew*), elimina la aparición de pulsos espurios (*glitches*) en las líneas de temporización, previene violaciones de tiempos de establecimiento y retención (*setup/hold*) por cruce entre dominios de reloj (CDC) y permite que el analizador estático de temporización (TimeQuest en Quartus Prime) garantice la operación fiable del circuito tanto a 1 s como a 100 ms.

Cuestionario

1. **¿Por qué un número con signo de 9 bits representa valores desde -256 hasta 255 y no desde -255 hasta 255?**
   Porque un registro de $n = 9$ bits tiene exactamente $2^9 = 512$ combinaciones binarias posibles. En complemento a dos el bit más significativo (`count[8]`) indica el signo (0 para no negativos y 1 para negativos). La mitad de las combinaciones ($2^8 = 256$ patrones, de `0_0000_0000` a `0_1111_1111`) se asigna al cero y a los enteros positivos (0 a 255). Como el cero solo tiene una representación única y ocupa una combinación del grupo con bit de signo en 0, las 256 combinaciones restantes con el bit de signo en 1 (`1_0000_0000` a `1_1111_1111`) representan exclusivamente números negativos, abarcando desde $-2^8 = -256$ hasta -1.

2. **¿Qué patrón binario representa -256 en complemento a dos de 9 bits?**
   El patrón binario es `1_0000_0000` (en hexadecimal `9'h100`), donde el bit de peso -2^8 vale 1 y todos los bits de magnitud del bit 7 al 0 valen 0: $-1 \cdot 2^8 + 0 = -256$.

3. **¿Qué ocurre al incrementar 255 y al decrementar -256?**
   Debido a que el contador opera con aritmética modular de 9 bits sin lógica de saturación, se produce un envolvimiento (*wrap-around*) en los extremos del rango representable:
   - Al incrementar 255 ($\texttt{0_1111_1111}_2 + 1$), el acarreo se propaga hasta el bit de signo generando $\texttt{1_0000_0000}_2$, que corresponde a -256.
   - Al decrementar -256 ($\texttt{1_0000_0000}_2 - 1$, equivalente a sumar $\texttt{1_1111_1111}_2$), se obtiene $\texttt{0_1111_1111}_2$, que corresponde a 255.

4. **¿Qué diferencia existe entre generar un reloj nuevo y generar un pulso de habilitación?**
   Generar un reloj nuevo consiste en usar la salida de un divisor o lógica combinacional para alimentar directamente la entrada de reloj (`clk`) de otros flip-flops, lo cual crea un nuevo dominio de reloj que no viaja por la red global dedicada de bajo sesgo de la FPGA, provocando desfases, posibles glitches, problemas de metastabilidad al comunicar señales entre dominios y dificultades en el análisis estático de temporización. En cambio, un pulso de habilitación es una señal síncrona que vale 1 durante un único periodo del reloj maestro (50 MHz) y se conecta a la entrada de habilitación (`CE`) de los flip-flops; así, todos los registros conmutan simultáneamente con el mismo reloj principal de 50 MHz.

5. **¿Por qué deben sincronizarse las entradas de los botones?**
   Porque el accionamiento de un botón o interruptor físico por parte del usuario es un evento totalmente asíncrono respecto al reloj interno de 50 MHz de la FPGA. Si la señal cambia dentro de la ventana crítica de tiempo de establecimiento (*setup time*) o de retención (*hold time*) del flip-flop receptor, este puede entrar en un estado donde hay un nivel de voltaje intermedio indefinido y oscilatorio. El sincronizador de dos flip-flops en cascada otorga un ciclo completo de reloj para que el primer flip-flop se estabilice antes de que el segundo capture el valor, reduciendo exponencialmente la probabilidad de propagar la metastabilidad hacia la FSM.

6. ¿Qué problema evita el circuito antirrebote?
Evita el fenómeno de rebote mecánico: cuando las láminas metálicas de un pulsador o interruptor hacen o rompen contacto, rebotan físicamente durante varios milisegundos generando decenas de transiciones eléctricas espurias ($1 \leftrightarrow 0$). A una frecuencia de muestreo de 50 MHz (periodo de 20 ns), un sistema digital sin filtro antirrebote interpretaría esos rebotes como múltiples pulsaciones consecutivas en lugar de una sola acción del usuario.

7. ¿Cuántos ciclos de 50 MHz corresponden a 100 ms?
Utilizando la relación $N = f_{\text{CLK}} \cdot T$:

$$N_{100\text{ ms}} = (50 \times 10^6 \text{ ciclos/s}) \times (0{,}1 \text{ s}) = 5{,}000{,}000 \text{ ciclos.}$$

Como el contador inicia desde 0, cuenta desde 0 hasta $N - 1 = 4{,}999{,}999$.

8. ¿Qué recursos adicionales pueden requerir los operadores de división y módulo durante la síntesis?
Cuando se sintetizan directamente operadores de división (`/`) y módulo (`%`) en Verilog —incluso entre constantes como 10 y 100 para extraer centenas, decenas y unidades—, el sintetizador genera redes puramente combinacionales de comparadores y restadores en cascada (arreglos divisores combinacionales). Esto consume una cantidad considerablemente mayor de LUTs / LEs y cadenas de acarreo, además de incrementar el retardo de propagación del camino crítico combinacional en comparación con algoritmos iterativos o de desplazamiento-suma como Double Dabble (Shift-and-Add-3).

Referencias

[1] "Two's complement," Wikipedia. [En línea]. Disponible: `https://en.wikipedia.org/wiki/Two%27s_complement`. [Consultado: 16-oct-2026].

[2] "Máquina de estados finitos" TheCodest. [En línea]. Disponible: `https://thecodest.co/es/diccionario/maquina-de-estados-finitos/`. [Consultado: 16-oct-2026].

[3] L. Llamas, "FPGA Debounce: How to Read Push Buttons Correctly." [En línea]. Disponible: `https://www.luisllamas.es/en/fpga-debounce-eliminar-rebotes/`. [Consultado: 16-oct-2026].

[4] "What is Debouncing in FPGA?," DEV Community. [En línea]. Disponible: `https://dev.to/carolineee/what-is-debouncing-in-fpga-40ke`. [Consultado: 16-oct-2026].

[5] "Metastability," FPGARelated.com. [En línea]. Disponible: `https://www.fpgarelated.com/glossary/metastability`. [Consultado: 16-oct-2026].

[6] "Clock Enable," FPGARelated.com. [En línea]. Disponible: `https://www.fpgarelated.com/glossary/clock-enable`. [Consultado: 16-oct-2026].

[7] Terasic Technologies, *DE10-Lite User Manual*. [En línea]. Disponible: `https://eecs.engineering.oregonstate.edu/education/inventory_datasheets/P1533579709.pdf`. [Consultado: 16-oct-2026].

[8] "Terasic Technologies DE10-Lite Board," Mouser Electronics. [En línea]. Disponible: `https://www.mouser.ca/terasic-de10-lite-board`. [Consultado: 16-oct-2026].